

PESQUISA: HOOKS REACT NATIVE

Hook Descrição

useState: Gerencia o estado local de um componente

useEffect: Executa código após renders e em resposta a mudanças

useRef: Guarda referências sem causar re-renderização

Q1. O que é um hook? Para que serve?

Hooks permitem a reutilização de recursos do React Native e lógicas sem precisar escrever uma classe, desse modo torna o código mais limpo reutilizável e fácil de gerenciar

Q2. Para cada um dos três hooks, explique: o que ele faz, quando você o usaria e escreva um exemplo mínimo de código.

- State Hook (useState):

Cria um estado local para guardar informações que podem ser mudadas, utilizamos ele quando queremos que algo do seu código mude quando o usuário interagir

```
import React, { useState } from 'react';
import { Text } from 'react-native';

export default function Exemple() {
  const [numero] = useState(10);

  return <Text>{numero}</Text>;
}
```

- Efeito Hook (useEffect):

Executa uma tarefa depois que o React Native “desenha” um componente (efeito colateral), utilizamos ele quando queremos fazer ações que podem afetar outros componentes.

```

import React, { useEffect } from 'react';
import { Text } from 'react-native';

export default function Example() {
  useEffect(() => {
    console.log('Componente montou');
  }, []);

  return <Text>Check no console!</Text>;
}

```

- Referência Hook (useRef):

Referencia um valor que não é necessário para a renderização e armazena informações persistentes, utilizamos ele em situações que precisamos manter uma informação entre renderizações.

```

import React, { useRef } from 'react';
import { Text } from 'react-native';

export default function App() {
  const ref = useRef('valor inicial');

  return <Text>{ref.current}</Text>;
}

```

Q3. O useEffect recebe um segundo parâmetro chamado array de dependências.

O que acontece quando esse array está vazio [] ? E quando não é passado nenhum array?

Quando não passamos nenhum array, por padrão, o React executa os efeitos após cada renderização do componente, o que inclui tanto a primeira renderização quanto todas as atualizações subsequentes.

Quando o array está vazio o efeito é executado **apenas uma vez**, logo após a primeira renderização.

Q4. Qual a diferença entre usar useRef e useState para guardar um valor? Em que situação do React Native o useRef é especialmente útil?

useState: É utilizado para informações que devem ser exibidas na tela ou que afetam a saída visual do componente, pois o React precisa atualizar a interface quando esses dados mudam.

useRef: É ideal para armazenar informações que não afetam a interface visual, funcionando como um "balde" para guardar valores que precisam persistir entre as renderizações sem interromper o fluxo do componente.

Q5. Quais são as duas regras dos hooks (Rules of Hooks)? Por que não podemos chamar um hook dentro de um if?

Regras:

- 1) Apenas chame Hooks no nível mais alto: Você não deve chamar Hooks dentro de condições.
- 2) Apenas chame Hooks de componentes funcionais: Os Hooks devem ser chamados apenas dentro de componentes funcionais do React ou de seus próprios Hooks customizados, e nunca em funções JavaScript comuns.

Não podemos chamar Hooks dentro de estruturas condicionais porque o React depende da ordem em que os Hooks são chamados para associar corretamente os estados e efeitos a cada renderização

Fontes:

- 1) <https://pt-br.react.dev/reference/react/useRef>
- 2) <http://pt-br.legacy.reactjs.org/docs/hooks-overview.html>